

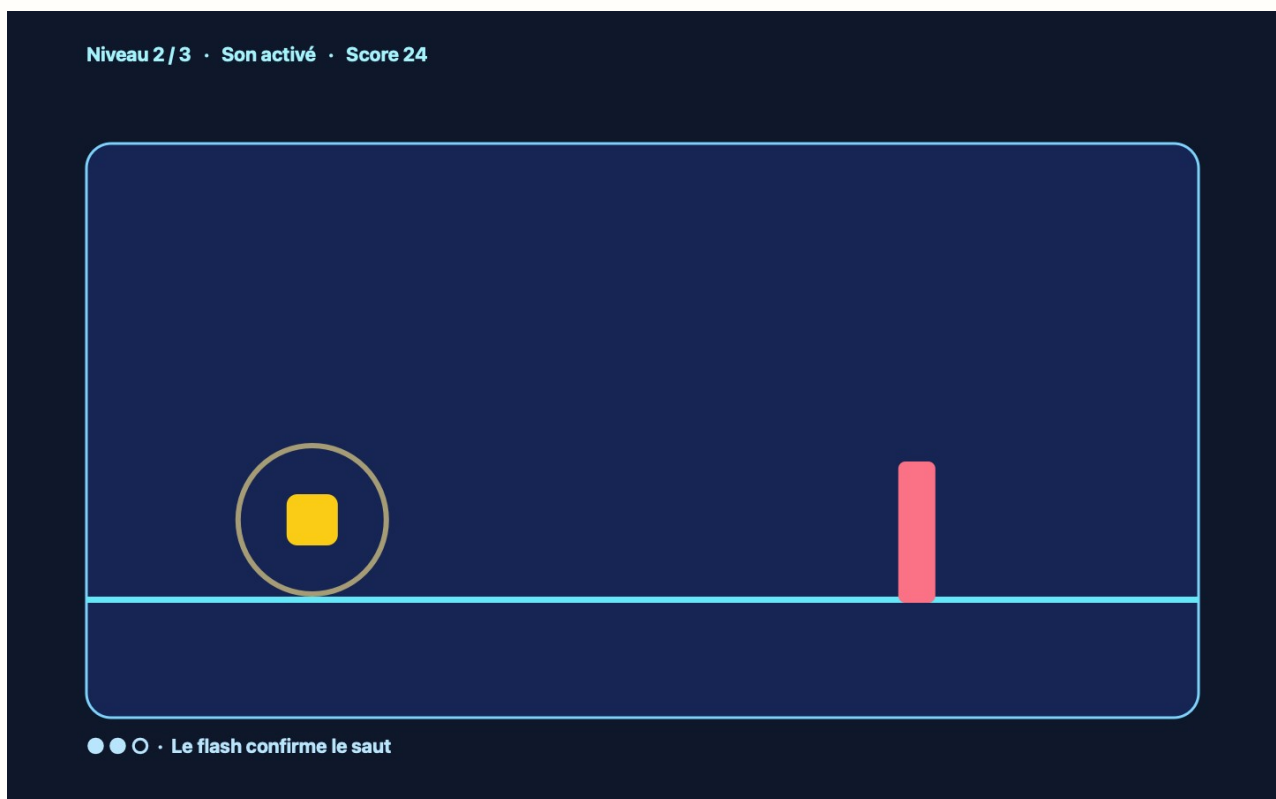
Atelier — Polish : faire sentir un événement

À ouvrir après 1-COURS.pdf du dossier projets/03-polish/. 2–3 périodes. Vous repartez du dossier m13/ qui possède déjà menu, jeu, over et un niveau terminé.

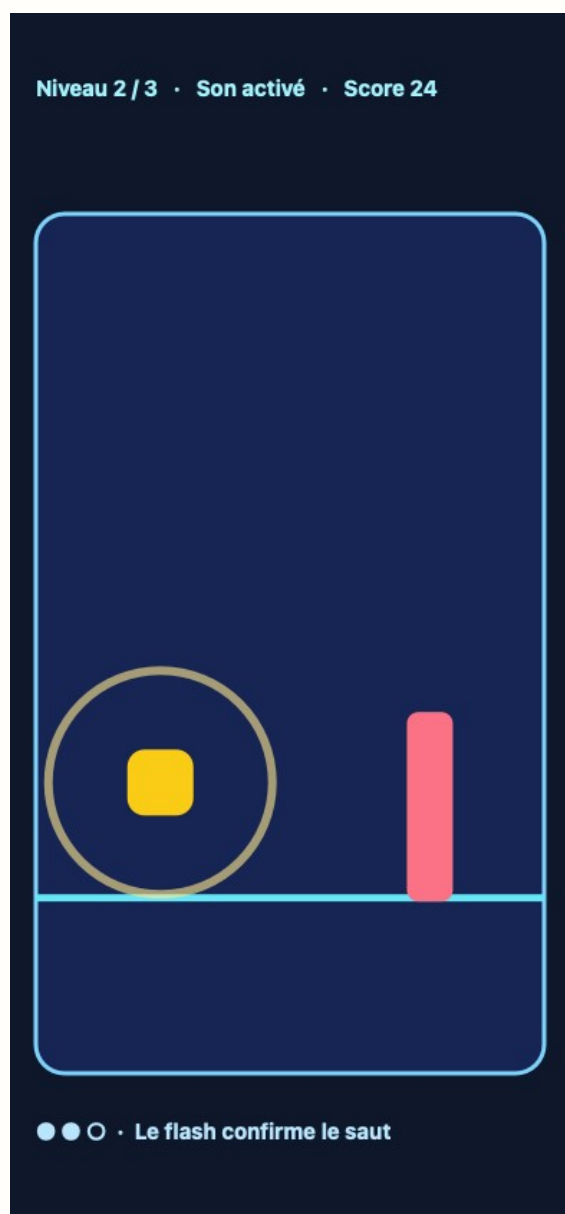
L'idée

Le polish confirme un événement du jeu; il ne remplace ni la règle ni l'état. Un flash bref peut confirmer le saut, un son court peut confirmer la défaite, et les niveaux peuvent changer par leurs données sans copier la boucle. La règle reste **un événement utile** → **un retour perceptible**.

À la fin, votre écran ressemble à ceci pendant le retour visuel du saut. Le niveau courant est visible, mais la mécanique du bouton n'a pas changé.



Sur un écran de 390 pixels, le niveau, le joueur et le retour visuel restent lisibles.



Vous gardez exactement cet arbre

```
m13/
├─ index.html
├─ concept.md
├─ README.md
├─ css/
│  └─ style.css
├─ images/
├─ sons/
│  └─ README.md
├─ js/
└─ jeu.js
```

sons/README.md indique la provenance de chaque son, ou précise aucun son externe. Un fichier téléchargé ne reste pas à la racine.

Étape 1 — Choisir un seul événement

Quand tout clignote et joue un son, rien n'informe vraiment la personne qui joue. Commencez par l'événement que vous voulez rendre impossible à manquer. Dans `concept.md`, ajoutez une seule ligne comme celle-ci :

```
Retour du saut : un anneau clair apparaît pendant 180 ms.
```

La phrase nomme l'événement, le retour et sa durée. Remplacez le saut par la défaite si c'est le moment le plus important de votre jeu; ne traitez pas les deux à la fois.

Étape 2 — Un premier retour visible

Votre action du bouton sait déjà appeler `joueur.sauter()`. Juste après cet appel, déclenchez une fonction `confirmerSaut()`. Son seul métier est d'ajouter une classe, puis de la retirer.

```
function confirmerSaut() {
  scene.classList.add("retour-saut");
  setTimeout(() => {
    /* à vous : retirer la classe retour-saut */
  }, 180);
}
```

Dans `css/style.css`, faites apparaître un contour ou un halo uniquement quand `#scene` porte cette classe. Testez dix sauts : le retour doit confirmer l'action sans cacher le joueur ni l'obstacle.

Étape 3 — Trois niveaux, une seule boucle

Copier la boucle trois fois crée trois versions qui se corrigent séparément. Les différences entre niveaux sont des données : vitesse, limite ou intervalle. Commencez avec un seul objet, puis observez-le dans la console.

```
const niveaux = [
  { vitesse: 2, limite: 320 },
  /* à vous : niveau 2, un seul paramètre plus difficile */
  /* à vous : niveau 3, un seul paramètre plus difficile */
];

let niveauActuel = 0;
```

Dans la boucle, lisez `niveaux[niveauActuel]`. Ne créez pas `boucleNiveau1`, `boucleNiveau2` et `boucleNiveau3`. La règle nommée **niveau = mêmes gestes, autres paramètres** doit rester visible dans le code.

Étape 4 — Passer au niveau suivant

Un niveau terminé change un index, pas tout le programme. Dans la branche de victoire, augmentez `niveauActuel` seulement s'il reste un objet dans `niveaux`, puis réinitialisez le joueur. Si le troisième

niveau est terminé, affichez un message de victoire distinct de la défaite.

Testez dans cet ordre : niveau 1, victoire, niveau 2. Avant de construire le niveau 3, affichez dans la console la vitesse ou la limite réellement lue par la boucle.

Étape 5 — Ajouter du son seulement s'il informe

Un son est utile quand il confirme un événement que l'image seule rend incertain. Si votre retour visuel suffit, gardez-le et écrivez `aucun son externe` dans `sons/README.md`. Sinon, ajoutez un seul son court, déclenché au même endroit que l'événement, puis nommez sa source et sa licence.

Ajoutez aussi un bouton Son activé / Son coupé. Le choix doit rester respecté jusqu'au prochain rechargement de la page.

Étape 6 — Publier ce que vous avez testé

Une URL n'est pas une nouvelle version du jeu : c'est la même version rendue accessible. Placez l'URL dans `README.md`, ouvrez-la, puis jouez jusqu'à `over` et jusqu'au niveau 2. Testez aussi la page à environ 390 pixels de large.

Ne publiez aucun son ni aucune image dont la licence n'est pas nommée. GitHub Pages ou itch.io conviennent; un moteur 3D et un store mobile restent hors de ce métier.

Réussi si

- `[]` `concept.md` nomme un événement, un retour et une durée.
- `[]` Le retour visuel ne change pas la règle du saut.
- `[]` Les trois niveaux vivent dans un tableau de données.
- `[]` La même boucle lit `niveaux[niveauActuel]`.
- `[]` La victoire et la défaite affichent des messages distincts.
- `[]` Le son est couplé à un événement et sa provenance est écrite, ou aucun son externe n'est utilisé.
- `[]` L'URL ouvre le jeu à 1440 et à 390 pixels.

AVEC L'IA

Montrez seulement le tableau `niveaux` et la ligne qui le lit. Demandez : « Quel paramètre change la difficulté, et quelle partie du code reste identique ? » Refusez toute proposition de nouvelle mécanique tant que les trois niveaux ne sont pas jouables.

POUR LES RAPIDES

Respectez `prefers-reduced-motion` : `reduce` : désactivez le halo animé, mais conservez un changement de couleur fixe. La confirmation reste perceptible sans mouvement.

Livrable

Rendez `m13/`, l'URL publique, `concept.md`, `README.md` et les licences éventuelles. À l'oral, montrez l'événement qui déclenche le polish et le tableau qui porte les différences entre niveaux.